

# Single Sign-On

Facilitate greater security, easier account management, and accelerated application onboarding and adoption.

Copy page

## Choose your integration approach

There are two ways to integrate Single Sign-On (SSO) with WorkOS:

### A With the standalone SSO API

The standalone API (covered in this document), is a standalone API for integrating into an existing auth stack.

### B Using WorkOS AuthKit

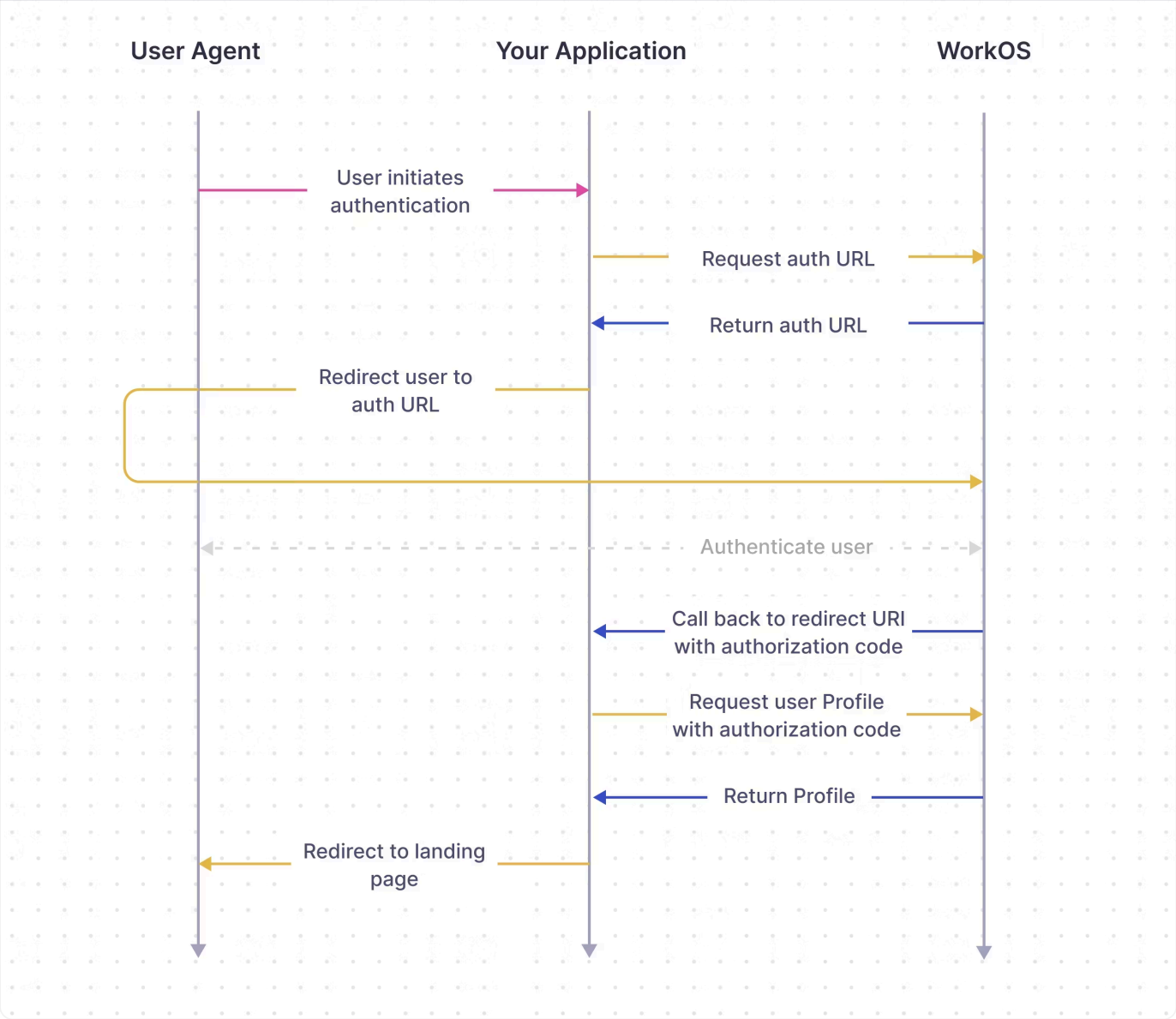
AuthKit is a complete authentication platform which includes SSO out of the box.

## How Single Sign-On works

Single Sign-On is the most frequently asked for requirement by organizations looking to adopt new SaaS applications. SSO enables authentication via an organization's identity provider (IdP).

This service is compatible with any IdP that supports either the SAML or OIDC protocols. It's modeled to meet the OAuth 2.0 framework specification, abstracting away the underlying authentication handshakes between different IdPs.

WorkOS Single Sign-On




WorkOS SSO API acts as authentication middleware and intentionally does not handle user database management for your application.

## What you'll build

In this guide, we'll take you from learning about Single Sign-On and POC-ing all the way through to authenticating your first user via the WorkOS SSO API.

## Before getting started

To get the most out of this guide, you'll need:

- A WorkOS account
- A local app to integrate SSO with.

WorkOS Single Sign-On

## API object definitions

**Connection**

The method by which a group of users (typically in a single organization) sign in to your application.

**Profile**

Represents an authenticated user. The Profile object contains information relevant to a user in the form of normalized and raw attributes.

## 1 Add SSO to your app

Let's build the SSO authentication workflow into your app.

### Install the WorkOS SDK

WorkOS offers native SDKs in several popular programming languages. Choose a language below to see instructions in your application's language.

Node.js | Ruby | Python
Go | PHP | Laravel
Java | .NET

Don't see an SDK you need? Contact us to request an SDK!

Install the SDK using the command below.

npm Yarn JavaScript

```
$ npm install @workos-inc/node
```

### Set secrets

To make calls to WorkOS, provide the API key and, in some cases, the client ID. Store these values as managed secrets, such as `WORKOS_API_KEY` and `WORKOS_CLIENT_ID`, and pass

WorkOS Single Sign-On

**Environment variables**

```
WORKOS_API_KEY='sk_example_123456789'
WORKOS_CLIENT_ID='client_123456789'
```

The code examples use your staging API keys when signed in

### Add an endpoint to initiate SSO

The endpoint to initiate SSO via the WorkOS API is responsible for handing off the rest of the authentication workflow to WorkOS. There are a couple configuration options shown below.

You can use the optional `state` parameter to encode arbitrary information to help restore application state between redirects.

**Using organization ID** | Using connection ID | Using provider

Use the organization parameter when authenticating a user by their specific organization. This is the preferred parameter for SAML and OIDC connections.

The example below uses the Test Organization that is available in your staging environment and uses a mock identity provider. It's created to help you test your SSO integration without having to go through the process of setting up an account with a real identity provider.

**Next.js** | Next.js (App Router) | Express — JavaScript

```
import type { NextApiRequest, NextApiResponse } from 'next';
import { WorkOS } from '@workos-inc/node';

const workos = new WorkOS(process.env.WORKOS_API_KEY);
const clientId = process.env.WORKOS_CLIENT_ID;

export default (_req: NextApiRequest, res: NextApiResponse) => {
  // Use the Test Organization ID to get started. Replace it with
  // the user's real organization ID when you finish the integration.
  const organization = 'org_test_idp';

  // The callback URI WorkOS should redirect to after the authentication
  const redirectUri = 'https://dashboard.my-app.com';

  const authorizationUrl = workos.sso.getAuthorizationUrl({
    organization,
```

```
20
21    res.redirect(authorizationUrl);
22  };
```

If there is an issue generating an authorization URL, WorkOS will return the redirect URI as is. Read the API Reference for more details.

### Add a callback endpoint

Next, let's add the redirect endpoint which will handle the callback from WorkOS after a user has authenticated with their identity provider. This endpoint should exchange the authorization code returned by WorkOS with the authenticated user's profile. The authorization code is valid for 10 minutes.

Next.js | Next.js (App Router) | Express — JavaScript

```
1  import type { NextApiRequest, NextApiResponse } from 'next';
2  import { WorkOS } from '@workos-inc/node';
3
4  const workos = new WorkOS(process.env.WORKOS_API_KEY);
5  const clientId = process.env.WORKOS_CLIENT_ID;
6
7  export default async (req: NextApiRequest, res: NextApiResponse) => {
8    const { code } = req.query;
9
10   const { profile } = await workos.sso.getProfileAndToken({
11     code,
12     clientId,
13   });
14
15   // Use the Test Organization ID to get started. Replace it with
16   // the user's real organization ID when you finish the integration.
17   const organization = 'org_test_idp';
18
19   // Validate that this profile belongs to the organization used for authentication
20   if (profile.organizationId !== organization) {
21     return res.status(401).send({
22       message: 'Unauthorized',
23     });
24   }
25
26   // Use the information in `profile` for further business logic.
27
28   res.redirect('/');
29 };
```

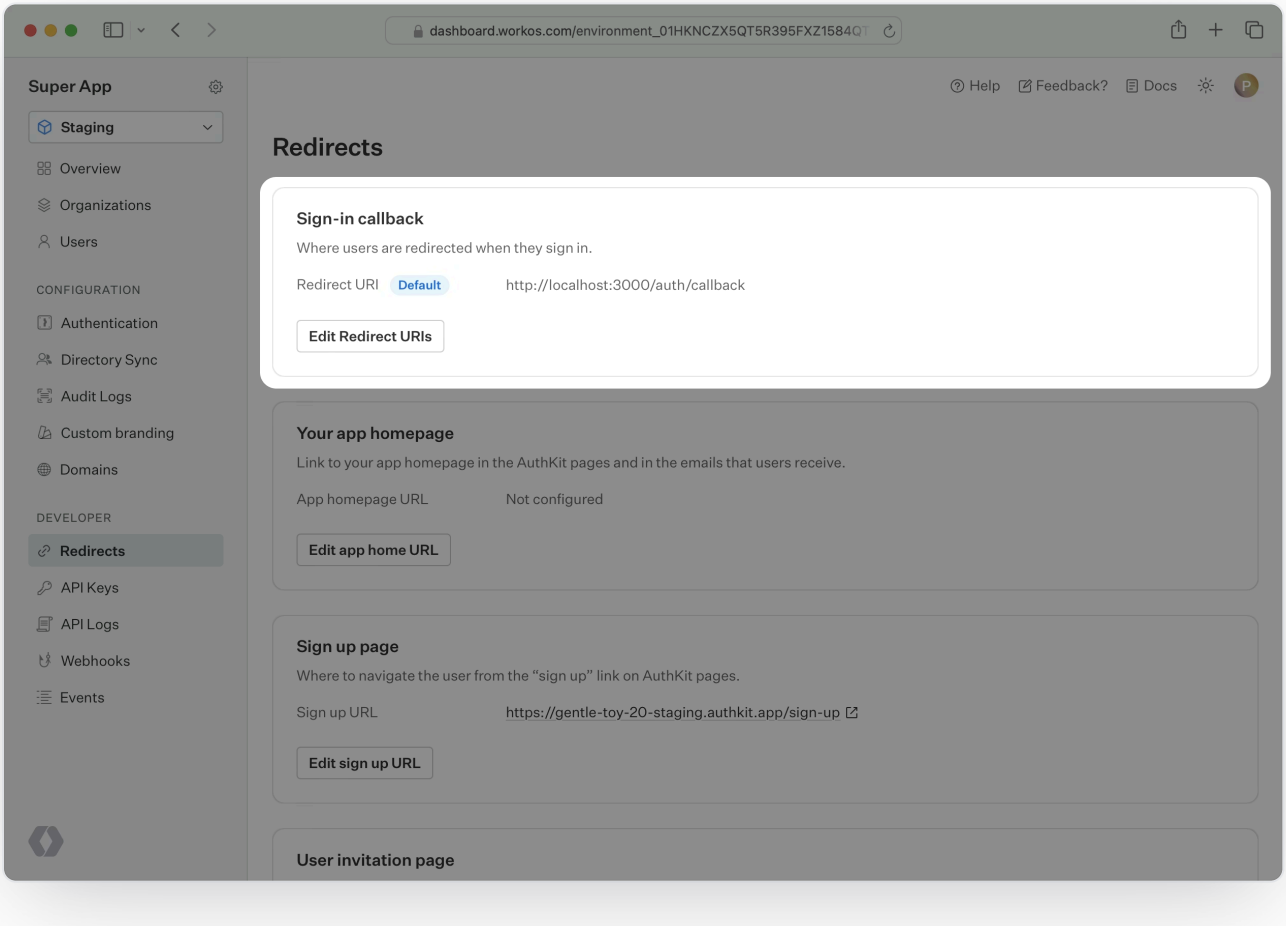
email addresses from outside their corporate domain (e.g. for guest users).

## 2 Configure a redirect URI

Go to the Redirects page in the dashboard to configure allowed redirect URIs. This is your callback endpoint you used in the previous section.

Multi-tenant apps will typically have a single redirect URI specified. You can set multiple redirect URIs for single-tenant apps. You'll need to be sure to specify which redirect URI to use in the WorkOS client call to fetch the authorization URL.

ⓘ More information about wildcard characters support can be found in the Redirect URIs guide.




### Identity provider-initiated SSO

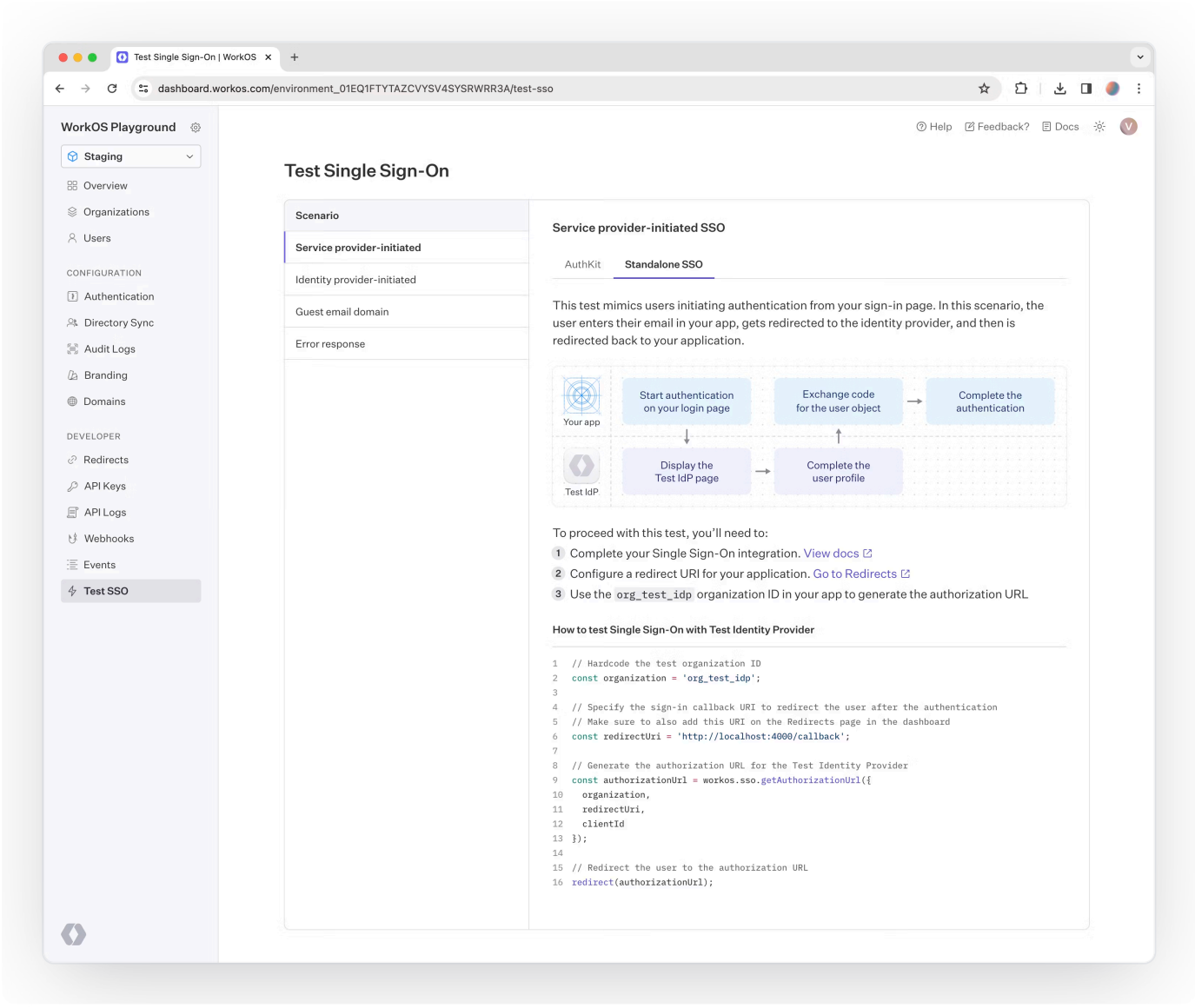
Normally, the default redirect URI you configure in the WorkOS dashboard is going to be used for all identity provider-initiated SSO sessions. This is because the WorkOS client is not

initiated sessions as a `RelayState` parameter in the SAML settings on their side.

Learn more about configuring IdP-initiated SSO in the Login Flows guide.

## 3 Test end-to-end

If you followed this guide, you used the Test Organization available in your staging environment to initiate SSO. With that, you can already test your integration end-to-end.




Head to the *Test SSO* page in the WorkOS Dashboard to get started with testing common login flows, or read on about that in detail in the next guide.

WorkOS Single Sign-On



**Features**

AuthKit

Single Sign-On

Directory Sync

Admin Portal

Fine-Grained Authorization

**Developers**

Documentation

Changelog

API Status

**Resources**

Blog

Podcast

Pricing

Security

Support

**Company**

About

Customers

Careers

Legal

Privacy

© WorkOS, Inc.

